

Video Transcript

Video 2: Using Time Loops to Show API Data (05:53)

Let's build a small simulation to explore the design decisions that you will need to make. As mentioned previously, the original tracking and the source of the data will be done by the MBTA and shared through their interfaces. When it comes to the front end, the client web app, we will provide you a basic version of that as well. So, your responsibility is that intermediate server, the one circle in red. So, to consider those choices and what you will be doing here, let's consider the following small simulation. We will be simulating the movement of four buses. As you can see there on the laptop, moving those buses across the screen.

We will have a time loop that will fire every 10 seconds. And every single time that fires, we will update the locations, and there are two files that we will share with you. The first one is `server.py`, the second one is `client`. Now we will also be able to use `cURL` at the command line to interrogate that API. Now before we move forward, it's worth mentioning that roughly speaking, we're working within that bounding box, which roughly captures the route that you will be working with. Now in addition to that, the data format that is expected by clients is the following.

You can see there that we have the ID of the bus, latitude and longitude, which is the location and the direction of the bus as well. Now let's take a look at those files and we'll start off with `server.py`. As you can see here, there are a number of imports, `timer`, `flask`, `time` again and `Json`. These correspond to being able to do the loop, a basic web server, some timestamps and some data conversions. As you can see here, the very first block of code is the original locations of the buses. The second part here has to do with the increments that are added on each time loop.

We have a function here that updates the location and finally here what we have is a time loop that fires every 10 seconds and updates those locations. Then at the bottom here of the file, we have the web server. Most of it is your default basics for a server. And then, we have here a route that sends you to the HTML file. We will leave that for later. And then, we have the block that can provide you with the location. As you can see there, it simply takes the data structure, which is `buses` and the updates that may have been done to it and returns it as a `json.dump`.

So, let's go ahead and run this. But before we do, let's take a look at what the client does. And as you can see here, the client is very simple. All it does is it sets up an address, then it uses `request.urlopen` to read what comes back from that address, that is that location we are using this part here, that is the route that we're using within the Flask server to share our data about the buses, and then we simply write it to the screen. Now just as in the server, we have a time loop that will be repeating every 10 seconds and asking the server once again, what is the location of the buses?

So, let's take a look at this and see how that runs. So, I'm going to start off here by running the server that is `server.py`. And this is up and running now. And now, I'm going to go ahead and fire up the client. And so this is simply `client.py`. And as you can see there, I've started to write to the console the timestamps. So, now I'm going to go ahead and fire up the client. And as you can see there, the client is getting the data. And let's go ahead and let it run a little bit more just to see that information update.

And if you take a closer look at each of those latitudes and longitudes, you will notice that they have changed. So, before you go forward, take a look at this code. Make sure you understand what's happening. Make sure you understand what the client is doing as well. And I just noticed that we left out `cURL`. Let's go ahead and access it through `cURL` simply to illustrate that application programming interface, that API. Now as you can see here that continues to run. I'm actually going to go ahead and stop the client and then I'm going to use `cURL` to head that address, it is going to be `localhost`. I'm running it in port 3000. And then, the path is `location`.

As you can see there, I get the location back just like I did when I was running the Python program. And I wanted to get it a little bit prettier here, but it doesn't really format any prettier. But if you look at the latitude and longitudes through each of those heads, depending on whether the data has been updated, I would have gotten a new set of coordinates. And after this, it should certainly be different, at least one of them. So, make sure you understand what's taking place here. This is a simulation, and so you can use it to explore. You're not hitting any external servers. It's all within your machine, although you already have the architectural boundaries that you will use later.